

Client-Side & Styling

Recap

Monday's patterns, then two new tools

File-Based Routing

```
src/routes/
├── +page.svelte           → /
├── +layout.svelte        → wraps all child pages
├── about/
│   └── +page.svelte      → /about
├── blog/
│   ├── +page.svelte     → /blog
│   └── [slug]/
│       └── +page.svelte  → /blog/hello-world (params.slug = 'hello-world')
```

- Every folder = a route, `+page.svelte` = the page
- `[slug]` = dynamic segment, available as `params.slug`
- `+layout.svelte` wraps all pages in its folder (and below)

Load Functions

`+page.server.ts` — fetch data on the server, pass it to the page

```
// src/routes/blog/[slug]/+page.server.ts
export const load: PageServerLoad = async ({ params }) => {
  const post = await getPost(params.slug);
  return { post };
};
```

```
<!-- src/routes/blog/[slug]/+page.svelte -->
<script lang="ts">
  let { data } = $props();
</script>

<h1>{data.post.title}</h1>
<p>{data.post.body}</p>
```

Form Actions

+page.server.ts — handle form submissions on the server

```
// src/routes/contact/+page.server.ts
export const actions: Actions = {
  default: async ({ request }) => {
    const data = await request.formData();
    const name = data.get('name') as string;
    // save to database, send email, etc.
    return { success: true };
  },
};
```

```
<!-- src/routes/contact/+page.svelte -->
<form method="POST">
  <input name="name" required />
  <button type="submit">Send</button>
</form>
```

redirect() and Cookies

Two new tools that complete the server-side picture

`redirect()` — send the user to a different URL from the server

```
import { redirect } from '@sveltejs/kit';

// In a load function or form action:
redirect(302, '/mpa/quiz/1'); // 302 = temporary redirect
redirect(303, '/mpa/quiz/done'); // 303 = redirect after POST (see-other)
```

Cookies — persist data across requests without a database

```
// In a load function or form action:
cookies.set('quiz-score', '5', { path: '/' });
const score = cookies.get('quiz-score'); // '5'
```

- Cookies travel with every request — available in every `load` and `action`
- Good for: preferences, session tokens, small state between pages

The Quiz App — MPA

State lives in the URL and in cookies

MPA Quiz: Architecture

Whiteboard

Client-Side State

What if the browser could think for itself?

Runes: `$state`

Declare reactive variables with `$state`

```
<script lang="ts">
  let count = $state(0);
</script>

<button onclick={() => count++}>
  Clicked {count} times
</button>
```

- Looks like normal JavaScript — Svelte tracks changes under the hood
- The UI updates automatically when `count` changes

\$props Are Reactive Too

`data` and `form` from `$props()` are `$state` under the hood

```

<!-- src/routes/contact/+page.svelte -->
<script lang="ts">
  let { data, form } = $props();
</script>

<h1>{data.title}</h1>

{#if form?.success}
  <p>Message sent!</p>
{/if}

```

- `data` updates when SvelteKit re-runs the `load` function (e.g., on navigation)
- `form` updates after a form action returns — no manual state management needed
- Any value from `$props()` is reactive — the UI re-renders when the parent passes new values

bind:value: Two-Way Binding

```
<script lang="ts">
  let name = $state('');
</script>

<input bind:value={name} placeholder="Your name" />
<p>Hello, {name}!</p>
```

- Keeps the input and the variable in sync
- Works with `<input>`, `<textarea>`, `<select>`

Other bindable attributes:

Element	Binding	Type
<code><input type="checkbox"></code>	<code>bind:checked</code>	boolean
<code><details></code>	<code>bind:open</code>	boolean
<code><video></code> / <code><audio></code>	<code>bind:currentTime</code> , <code>bind:paused</code>	number, boolean

\$derived: Computed Values

Values that update automatically when dependencies change

```
<script lang="ts">
  let price = $state(100);
  let quantity = $state(1);
  let total = $derived(price * quantity);
</script>

<input type="number" bind:value={price} />
<input type="number" bind:value={quantity} />
<p>Total: {total} EUR</p>
```

- `$derived.by(() => { ... })` for multi-line logic

\$effect

Side effects that run when dependencies change

```
<script lang="ts">
  let query = $state('');

  $effect(() => {
    console.log('Search query changed to:', query);
    // Could save to localStorage, trigger an API call, etc.
  });
</script>

<input bind:value={query} placeholder="Search ... " />
```

- `$inspect(count)` — dev-only debug logging (stripped in production)

Mental Model

Rune	Purpose	Analogy
<code>\$state</code>	Input / source of truth	A variable that notifies on change
<code>\$derived</code>	Computed output	A spreadsheet formula
<code>\$effect</code>	Side effect	An event listener

Rule of thumb: prefer `$derived` over `$effect` whenever possible

Fine-Grained Reactivity

Level 1 — Subscribers: Svelte tracks which `{expression}`, `$derived`, and `$effect` reads which `$state`. When a value changes, only the subscribers that use it re-run — not the whole component.

Level 2 — Properties: tracking goes deeper than “the variable changed”:

- `$state([a, b, c])` — each array element is tracked separately. An `$effect` reading `items[0]` won't re-run when `items[2]` changes.
- `$state({ name, email })` — each field is tracked separately. Reading `user.name` won't react to `user.email` changing.

The mechanism: `$state` creates a **signal** — a value with a subscriber list. The compiler rewrites variable access into function calls (getters/setters), but subscribers are tracked at runtime — when code runs and reads a signal, it registers itself.

Exercise: Interactive Widget

Build a small interactive component using runes.

Pick one (or invent your own):

- **Unit converter** — input value + units, `$derived` result
- **Tip calculator** — bill + tip %, derived total and per-person split
- **Todo list** — `$state` array, `$derived` remaining count, add/remove/toggle
- **BMI calculator** — height + weight, `$derived.by` for BMI + category

Requirements: use `$state`, `$derived`, and `bind:value`

Quiz App (SPA)

Same app, same data — different architecture

API Route: `+server.ts`

We need a way for the client to get the questions — a JSON endpoint

```
// src/routes/api/questions/+server.ts
import { json } from '@sveltejs/kit';
import { getQuestions } from '$lib/server/quiz-service';
import type { RequestHandler } from '.$types';

export const GET: RequestHandler = async () => {
  const questions = getQuestions();
  const payload = questions.map((q) => ({
    question: q.question,
    correct: q.correct,
    answers: shuffle([q.correct, ...q.wrong]),
  }));
  return json(payload);
};
```

- `+server.ts` = API endpoint (returns JSON, not HTML)
- Export named handlers: `GET`, `POST`, `PUT`, `DELETE`
- Visit `/api/questions` in the browser to see the raw JSON

When to Use API Routes

Use Case	Approach
Initial page data	<code>+page.server.ts</code> load function
Form submissions	Form actions in <code>+page.server.ts</code>
Client-side dynamic fetching	<code>+server.ts</code> API route
External integrations (webhooks)	<code>+server.ts</code> API route
Mobile app backend	<code>+server.ts</code> API route

Client-Side Quiz: The Script

Everything that was in the URL and cookies now lives in `$state`

```
<script lang="ts">
  let questions = $state([]);
  let userAnswers = $state<string[]>([]);

  let currentIndex = $derived(userAnswers.length);
  let currentQuestion = $derived(questions[currentIndex]);
  const isCorrect = (i: number) => userAnswers[i] === questions[i]?.correct;
  let correctCount = $derived(userAnswers.filter((_, i) => isCorrect(i)).length);
  let isComplete = $derived(questions.length > 0 && currentIndex ≥ questions.length);

  $effect(() => {
    fetch('/api/questions').then(r => r.json()).then(data => {
      questions = data;
    });
  });
</script>
```

Where Does State Live Now?

State	MPA	Client-Side
Current question	URL (<code>/mpa/quiz/3</code>)	<code>\$derived</code> → <code>userAnswers.length</code>
Score	Cookie (<code>quiz-score</code>)	<code>\$derived</code> → <code>correctCount</code>
Answer choices	Server memory	<code>\$state([])</code> → fetched once
Navigation	<code>redirect(303, ...)</code>	<code>userAnswers.push(answer)</code>

Key shift: state moved from the platform (URL, cookies, HTTP) into JavaScript variables

MPA vs. SPA: Trade-Offs

MPA

- Standards-compliant (plain HTML/HTTP)
- Works without JavaScript
- State is shareable/bookmarkable (lives in URL)
- Survives browser close (cookies, URL)
- Back/forward navigation works naturally

SPA

- Instant feedback, no network round-trip (works offline)
- State is local, easier to reason about
- Enables complex interactions (drag & drop, animations)

Neither is "better" — choose based on your requirements. Real apps mix both.

Break

Project Discussion

Project Discussion

Each student presents their project idea:

1. **What does the app do?** (one sentence)
2. **What data does it store?** (tables/fields)
3. **Who can do what?** (auth, public vs. private)

Discussion points:

- Is the scope right for ~14 hours of work?
- Can lean server-side (CRUD, form actions) or client-side (reactive UI)
- Students with a clear idea can start building during tomorrow's exercises

Components & Props

Breaking a page into reusable pieces

Components & Props

- **When to extract:** repeated UI, logical grouping, or file too long
- Place components in `src/lib/components/`

```

<!-- src/lib/components/Card.svelte -->
<script lang="ts">
  let { title, description } = $props();
</script>

```

```

<div class="card">
  <h3>{title}</h3>
  <p>{description}</p>
</div>

```

```

<script lang="ts">
  import Card from '$lib/components/Card.svelte';
</script>

```

```

<Card title="Hello" description="This is a card component." />

```

Snippets, `.svelte.ts`, Context API

Three tools for when components get complex — brief overview

Snippets — named, reusable template blocks

```
{#snippet greeting(name)}  
  <p>Hello, {name}!</p>  
{/snippet}  
  
{@render greeting('World')}
```

`.svelte.ts` **files** — use runes outside components (regular `.ts` files can't)

```
// src/lib/quiz-store.svelte.ts — can use $state, $derived, etc.  
export class QuizStore { questions = $state([]); ... }
```

Context API — share state without prop drilling

```
// Parent: setContext('quiz', new QuizStore());  
// Child:  const quiz = getContext<QuizStore>('quiz');
```

Tailwind CSS

Utility-first styling

Tailwind Setup

```
bunx sv add tailwindcss
```

Card example:

```
<div class="max-w-sm rounded-lg border border-gray-200 bg-white p-6 shadow-sm">
  <h3 class="mb-2 text-xl font-bold text-gray-900">Card Title</h3>
  <p class="text-gray-600">Some description text here.</p>
  <button class="mt-4 rounded bg-blue-600 px-4 py-2 text-white hover:bg-blue-700">
    Action
  </button>
</div>
```

Want pre-built components? [daisyUI](#) — Tailwind component library (`bun add -D daisyui`)

Core Classes

Category	Examples
Spacing	<code>p-4</code> , <code>m-2</code> , <code>gap-4</code>
Sizing	<code>w-full</code> , <code>max-w-lg</code> , <code>h-screen</code>
Flexbox	<code>flex</code> , <code>flex-col</code> , <code>items-center</code> , <code>justify-between</code>
Colors	<code>text-gray-900</code> , <code>bg-blue-600</code> , <code>border-gray-200</code>
Responsive	<code>sm:</code> , <code>md:</code> , <code>lg:</code> prefixes (mobile-first)

`class:` **directive** — toggle classes based on state:

```
<button class="px-4 py-2 rounded" class:bg-blue-600={active} class:bg-gray-300={!active}>
  {active ? 'On' : 'Off'}
</button>
```

Layout + Semantic HTML Tips

```
<!-- src/routes/+layout.svelte -->
<div class="flex min-h-screen flex-col">
  <header class="border-b bg-white px-6 py-4">
    <nav class="flex items-center gap-6">
      <a href="/" class="text-lg font-bold">MyApp</a>
      <a href="/about" class="text-gray-600 hover:text-gray-900">About</a>
    </nav>
  </header>
  <main class="flex-1 p-6">
    {@render children()}
  </main>
  <footer class="border-t px-6 py-4 text-sm text-gray-500">Built with SvelteKit</footer>
</div>
```

- Use `<header>`, `<nav>`, `<main>`, `<footer>` — not `<div>` soup
- `<button>` for actions, `<a>` for navigation (never `<div onclick>`)
- One `<h1>` per page, heading hierarchy `<h1>` → `<h6>`
- **Why:** accessibility (screen readers), SEO, maintainability

Transitions

Animate elements entering and leaving the DOM

```
<script lang="ts">
  import { fade, fly, slide } from 'svelte/transition';
  let visible = $state(true);
</script>

<button onclick={() => visible = !visible}>Toggle</button>

{#if visible}
  <p transition:fade>Fades in and out</p>
  <p in:fly={{ y: 20 }} out:fade>Flies in, fades out</p>
  <p transition:slide>Slides open and closed</p>
{/if}
```

- `transition:` = both in and out; `in:` / `out:` = separate
- Built-in: `fade`, `fly`, `slide`, `scale`, `blur`, `draw`

Attachments

Run code when an element is added to the DOM (replaces `use:` actions)

```
<script lang="ts">
  function autofocus(element: HTMLElement) {
    element.focus();
    return () => { /* cleanup */ };
  }
</script>

<input {@attach autofocus} />
```

- Return a cleanup function for teardown (like `$effect`)
- Reactive: re-runs when dependencies change

Fonts

```
/* src/app.css */  
@import url('https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&display=swap');  
  
@theme {  
  --font-sans: 'Inter', sans-serif;  
}
```

Browse fonts at: <https://fonts.google.com>

Icons

Iconify — thousands of icon sets via CSS classes (Tailwind 4 plugin)

```
bun add -D @iconify/tailwind4 @iconify/json
```

```
/* src/app.css */  
@plugin "@iconify/tailwind4";
```

```
<span class="iconify lucide--home"></span>  
<span class="iconify lucide--settings"></span>
```

Browse icons at: <https://icones.js.org>

Exercise: Style the Project

Apply Tailwind to your existing project.

Tasks:

1. Install Tailwind: `bunx sv add tailwindcss`
2. Rework `+layout.svelte` with `<header>` / `<nav>` / `<main>` / `<footer>`
3. Style at least one page with Tailwind utilities
4. Add a custom font (Google Fonts or system font stack)
5. Add icons to navigation or buttons (Iconify)

Wrap-Up

Today we covered:

- `redirect()` and cookies — completing the server toolkit
- MPA quiz: state lives in the URL and cookies, no client JS needed
- Runes: `$state` , `$derived` , `$effect`
- Client-side quiz: same app, state in JavaScript variables
- API routes (`+server.ts`) for JSON endpoints
- Components, props, snippets (brief)
- Tailwind CSS, fonts, and icons

The key insight: the MPA→SPA shift is about *where state lives*

Tomorrow: databases, authentication, and deployment

Come with questions about your project idea!